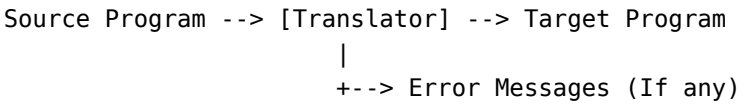
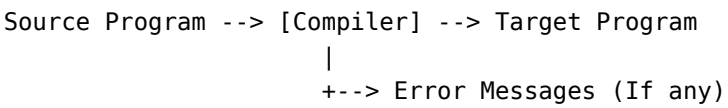
[SLIDE ORGANIZATION]
Course: CPT316 Programming Language Implementation and Paradigms.
Topic 1: Introduction to Compiler Construction.
Background: overlapping translucent geometric polygons.
These shapes are decorative, not a data-flow or syntax diagram.
No content-bearing diagram is present.

[TABLE OF CONTENTS / ORDERED SECTION HIERARCHY]
Topic 1 contains six sections in this order:
1. Structure and Phases of a Compiler
2. Analysis Synthesis model of compilation
3. Phases of compiler
4. Context of Compiler (Cousins of compiler)
5. Pass structure
6. Types of compiler
No inter-section arrows or dependencies are shown.

[SECTION NAVIGATION - CURRENT TOPIC]
Six-item Topics outline from source slide 2.
Highlighted item: 1. Structure and Phases of a Compiler.
The pink fill marks the active section, not a separate entity.
Remaining five items are unhighlighted.



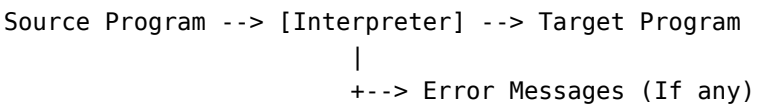
Meaning: the translator receives a source program and produces a corresponding target program; errors are a possible secondary output.
The arrow to Error Messages points DOWN from the Translator.
The numbered list names 3 kinds of translators:
(1) Compiler, (2) Interpreter, (3) Assembler.



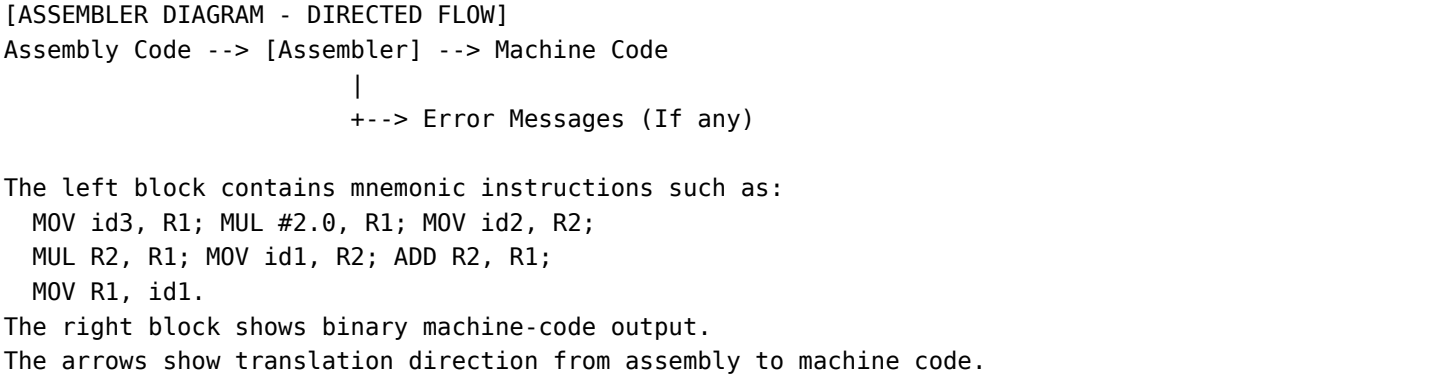
Source code shown in the left document:

```
void main() {
  int a=1,b=2,c;
  c=a+b;
  printf("%d",c);
}
```

Target document is represented by binary digits.
The compiler acts on the source program as input and emits
corresponding target-code output (and possible diagnostic messages).

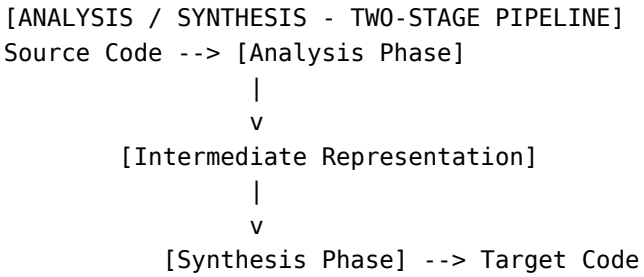


The phrase "line by line" appears OVER the rightward processing connection. It qualifies how the interpreter handles source code. Input is shown as a small C-like program; the right document shows rows of binary digits as an illustration of target output. This is the slide's conceptual drawing, not a universal claim that all interpreters generate a separate binary target file.



The left block contains mnemonic instructions such as:
MOV id3, R1; MUL #2.0, R1; MOV id2, R2;
MUL R2, R1; MOV id1, R2; ADD R2, R1;
MOV R1, id1.
The right block shows binary machine-code output.
The arrows show translation direction from assembly to machine code.

[SECTION NAVIGATION - CURRENT TOPIC]
The same six-item Topic 1 outline appears here.
Highlighted item: 2. Analysis Synthesis model of compilation.
Highlighted row represents the current lecture section.



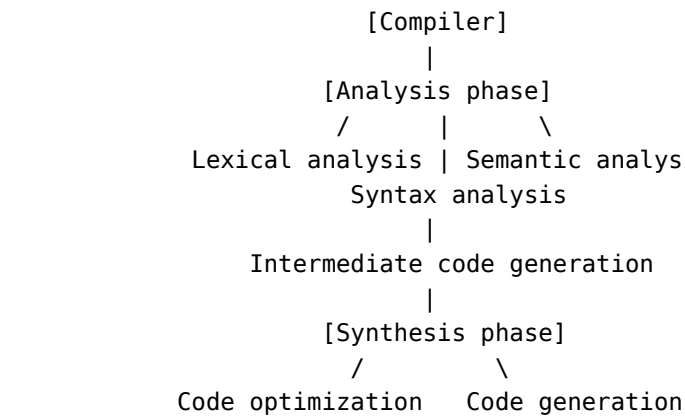
The source document contains a C-like program; the target document shows binary digits. The intermediate representation is a bridge: analysis extracts/structures meaning; synthesis constructs output. In the original graphic, directed arrows connect the stages in this logical sequence even though boxes are arranged in a zigzag.

[PHASE MEMBERSHIP / GROUPING]
Analysis Phase comprises:
  1. Lexical analysis
  2. Syntax analysis
  3. Semantic analysis
Output of analysis = Intermediate Representation.

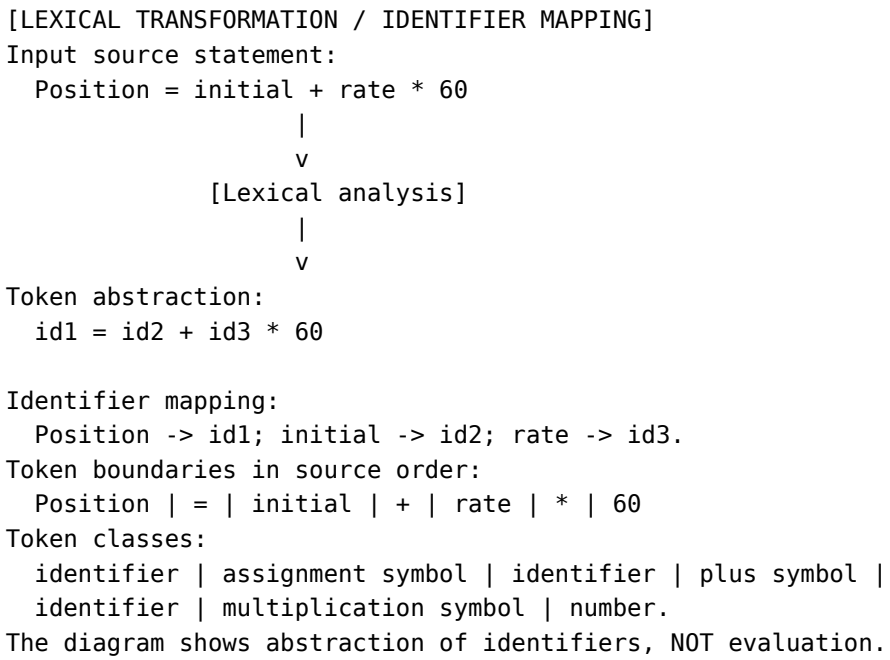
Synthesis Phase comprises:
  1. Code optimization
  2. Code generation
Input to synthesis = Intermediate Representation.

Relationship:
Source program -> Analysis -> Intermediate representation
               -> Synthesis -> Target program.

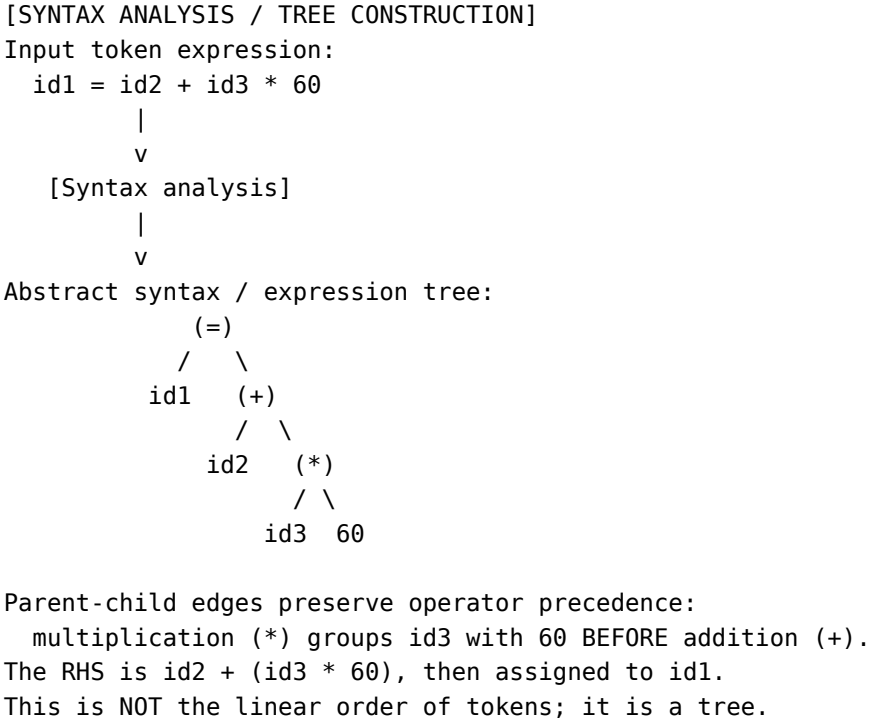
[SECTION NAVIGATION - CURRENT TOPIC]
Six-item Topics outline; section highlighted:
3. Phases of compiler.
No arrows or special graph relationships are on this slide.



The phase group labels form a hierarchy; arrows point from analysis outputs toward intermediate code and then synthesis.
Sequential processing order is:
Lexical -> Syntax -> Semantic -> Intermediate code -> Code optimization -> Code generation.
Pink highlighted node: Lexical analysis.


[LEXICAL TRANSFORMATION / IDENTIFIER MAPPING]
Input source statement:
Position = initial + rate * 60
[Lexical analysis]
Token abstraction:
id1 = id2 + id3 * 60
Identifier mapping:
Position -> id1; initial -> id2; rate -> id3.
Token boundaries in source order:
Position | = | initial | + | rate | * | 60
Token classes:
identifier | assignment symbol | identifier | plus symbol |
identifier | multiplication symbol | number.
The diagram shows abstraction of identifiers, NOT evaluation.

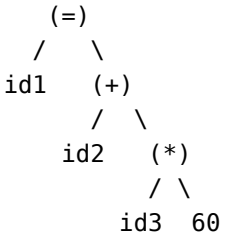
[COMPILER PHASE HIERARCHY - FOCUS CHANGE]
Same directed phase graph as source slide 12:
Lexical analysis -> Syntax analysis -> Semantic analysis ->
Intermediate code generation -> Code optimization -> Code generation.
Analysis group = lexical + syntax + semantic.
Synthesis group = optimization + code generation.
Pink highlighted node: Syntax analysis.
All other phase nodes remain present.



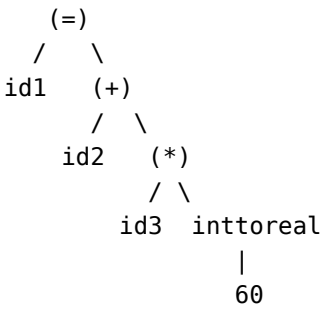
Parent-child edges preserve operator precedence:
multiplication (*) groups id3 with 60 BEFORE addition (+).
The RHS is id2 + (id3 * 60), then assigned to id1.
This is NOT the linear order of tokens; it is a tree.

[COMPILER PHASE HIERARCHY - FOCUS CHANGE]
Same compiler phase hierarchy shown on slides 12 and 14.
Pink highlighted node: Semantic analysis.
Context: Semantic analysis follows syntax analysis and
precedes intermediate code generation in the example.

[SEMANTIC ANALYSIS / TYPE COERCION IN TREE]
Before semantic analysis, the parse tree is:



Given: id1, id2, and id3 are REAL-valued identifiers.
The literal 60 is an INTEGER. Semantic analysis inserts a
conversion node above the literal in the multiplication branch:



Meaning: convert integer literal 60 to real before multiplying
by the real-typed id3. Other operator/tree edges are retained.

[COMPILER PHASE HIERARCHY - FOCUS CHANGE]
Same directed phase graph as slide 12.
Pink highlighted node: Intermediate code generation.
Location: after semantic analysis and before code optimization.
The text label "Intermediate code generation" names the bridge
between the analysis and synthesis groups in this slide's model.

```
[INTERMEDIATE CODE / TREE-TO-THREE-ADDRESS FLOW]
Typed expression tree from slide 17 has structure:
  id1 = id2 + (id3 * inttoreal(60))

Post-order decomposition into temporary results:
  t1 = inttoreal(60)     [conversion of literal]
  t2 = id3 * t1          [multiplication result]
  t3 = t2 + id2          [addition result]
  id1 = t3               [assignment result]

Dependencies:
  60 -> inttoreal -> t1
  (id3, t1) -> multiply -> t2
  (t2, id2) -> add -> t3
  t3 -> assign -> id1

The colored boxes t1, t2, t3 denote names for intermediate
results, not extra source-language variables.
```

[COMPILER PHASE HIERARCHY - FOCUS CHANGE]
Same directed phase graph as source slide 12.
Pink highlighted node: Code optimization.
Synthesis Phase contains Code optimization and Code generation.
Pipeline context: intermediate code -> code optimization ->
code generation.

```
[CODE OPTIMIZATION / BEFORE -> AFTER]
Input "Intermediate code" (top box):
  t1 = inttoreal(60)
  t2 = id3 * t1
  t3 = t2 + id2
  id1 = t3
           |
           v
      [Code optimization]
           |
           v
Output "Code optimization" (bottom):
  t1 = id3 * 60.0
  id1 = id2 + t1

Optimization replaces integer-to-real conversion with the real
literal 60.0, removes some temporaries, and preserves the
expression's result under the example's assumptions.
The arrows mean transformed code, NOT additional program execution.
```

[COMPILER PHASE HIERARCHY - FOCUS CHANGE]
Same directed phase graph as slide 12.
Pink highlighted node: Code generation.
It follows code optimization and produces target instructions.

```
[CODE GENERATION / INSTRUCTION DEPENDENCIES]
Optimized intermediate code:
  t1 = id3 * 60.0
  id1 = id2 + t1
                |
                v
        [Code generation]
                |
                v
Machine-like instruction sequence:
  MOV id3, R2       # R2 <- id3
  MUL #60.0, R2     # R2 <- R2 * 60.0
  MOV id2, R1       # R1 <- id2
  ADD R2, R1        # R1 <- R1 + R2
  MOV R1, id1       # id1 <- R1

Register association in red source note:
  id3 -> R2; id2 -> R1.
The order preserves the multiply-before-add dependency.
```

[WHOLE COMPILER / DIRECTED PROCESS FLOW]
Source program
  -> Lexical analysis   [Analysis Phase]
  -> Syntax analysis   [Analysis Phase]
  -> Semantic analysis   [Analysis Phase]
  -> Intermediate code   [IR boundary]
  -> Code optimization   [Synthesis Phase]
  -> Code generation   [Synthesis Phase]
  -> Target Program

[CROSS-CUTTING CONNECTIONS, NOT SEQUENTIAL STAGES]
Symbol table is connected by drawn edges to EVERY central stage:
lexical, syntax, semantic, intermediate, optimization, generation.
Error detection and recovery is ALSO connected to those stages.
Do not interpret symbol table or error handling as just one
additional step after the compiler pipeline.

[SYMBOL TABLE - THREE COLUMNS]
Variable Name | Type | Address
Position | Float | 0001
Initial | Float | 0005
Rate | Float | 0009

The downward symbol-table arrow points to this table of entries.

[FRONT END VS BACK END - GROUPING]
FRONT END (primarily source-language-dependent):
 - Lexical analysis
 - Syntax analysis
 - Semantic analysis
 - Intermediate code generation
 - Creation of symbol table

BACK END (primarily target-machine-dependent):
 - Code optimization
 - Code generation phase
 - Error handling and symbol table operation

These are two GROUPS of compiler work, not two data entities.
The grouping is as stated by the lecture; actual compiler
implementations may move some services between front and back.

[COMPILER VS INTERPRETER - ROW-ALIGNED COMPARISON]
Row 1, handling unit:
  Compiler: scans and translates whole program.
  Interpreter: translates one statement at a time.
Row 2, intermediate code (as the slide claims):
  Compiler: generates intermediate code.
  Interpreter: does not generate intermediate code.
Row 3, error reporting (as the slide claims):
  Compiler: errors reported after whole program checked.
  Interpreter: error reported per interpreted instruction.
Row 4, memory (as the slide claims):
  Compiler: more.   Interpreter: less.
Row 5, examples:
  Compiler: C compiler.
  Interpreter: Basic, Python, Ruby.

CAUTION: These are course-slide generalizations. Interpreters
may internally compile to intermediate code / bytecode, and
error-reporting behavior is implementation-dependent.

[SECTION NAVIGATION - CURRENT TOPIC]
Six-item Topics outline.
Highlighted item: 4. Context of Compiler (Cousins of compiler).
The selection indicator is not part of the execution pipeline.

[COMPILER CONTEXT / EXTERNAL TOOLCHAIN]
Skeletal Source Program
  -> [Preprocessor]
  -> Source Program
  -> [Compiler]
  -> Target Assembly Program
  -> [Assembler]
  -> Relocatable Object Code
  -> [Linker / Loader] <-- Libraries & Object Files
  -> Absolute Machine Code

Arrows are directed DOWN the main chain.
The Libraries & Object Files box feeds INTO the Linker / Loader
FROM THE LEFT (it is not produced by it).
The named cousin tools are Preprocessor, Assembler, Linker, Loader.

[TOOLCHAIN / PREPROCESSOR HIGHLIGHTED]
Skeletal Source Program -> [Preprocessor] -> Source Program
  -> [Compiler] -> Target Assembly Program
  -> [Assembler] -> Relocatable Object Code
  -> [Linker / Loader; + Libraries & Object Files]
  -> Absolute Machine Code

Cyan highlighted node: Preprocessor (top stage).
Its tasks listed on this slide:
1. Macro processing (e.g., #define PI ...)
2. File inclusion (e.g., #include<stdio.h>)
3. "Rational preprocessor" constructs (source wording)
4. Language extensions (example: embedded database language)
Highlighted tool is within the pipeline; no new branch is added.
Note: the label "Rational preprocessor" is copied from the
source, not silently corrected.

[TOOLCHAIN / COMPILER HIGHLIGHTED]
Skeletal Source Program -> Preprocessor -> Source Program
  -> [Compiler] -> Target Assembly Program
  -> Assembler -> Relocatable Object Code
  -> Linker / Loader (+ Libraries & Object Files)
  -> Absolute Machine Code

Cyan highlighted node: Compiler, second transformation step.
Input edge carries Source Program; output edge carries
Target Assembly Program in THIS pictured example.

[TOOLCHAIN / ASSEMBLER HIGHLIGHTED]
Skeletal Source Program -> Preprocessor -> Source Program
  -> Compiler -> Target Assembly Program
  -> [Assembler] -> Relocatable Object Code
  -> Linker / Loader (+ Libraries & Object Files)
  -> Absolute Machine Code

Cyan highlighted node: Assembler, third transformation step.
Input edge: Target Assembly Program.
Output edge: Relocatable Object Code.

[TOOLCHAIN / LINKER AND LOADER HIGHLIGHTED]
Skeletal Source Program -> Preprocessor -> Source Program
  -> Compiler -> Target Assembly Program
  -> Assembler -> Relocatable Object Code
  -> [Linker / Loader] <-- Libraries & Object Files
  -> Absolute Machine Code

Cyan highlighted node: Linker / Loader, final pictured step.
Linker combines code from several relocatable object files and
libraries into one program. Loader relocates addresses and places
instructions/data at appropriate memory locations.
The diagram combines linker and loader into ONE box, but the
slide's prose discusses them as distinct programs.

[SECTION NAVIGATION - CURRENT TOPIC]
Six-item Topics outline.
Highlighted item: 5. Pass structure.
The highlight merely marks the new section.

[PASS STRUCTURE / TEMPORAL ORDER]
Definition: a "pass" is one complete scan through the program.
ONE-PASS structure:
  read/analyze source statement -> synthesize corresponding output
  (for each statement in the same pass)
TWO-PASS structure:
  Pass 1: analysis -> intermediate information
  Pass 2: use intermediate information -> target output
A forward reference can make immediate one-pass code generation
difficult, because a referenced definition may not be known yet.

[TWO-PASS SOLUTION TO FORWARD REFERENCES]
A forward reference means a use appears BEFORE a later
definition in the source order.

Pass I (first scan):
  Source program -> analyze -> record relevant information
                (e.g., declarations, symbols, addresses)

Pass II (second scan):
  Source program + recorded information -> generate target code

The two horizontal bars "Pass I" and "Pass II" introduce
SEPARATE operations; they are not branches run in parallel.

[TRADEOFF - REDUCING NUMBER OF PASSES]
Fewer passes:
  + reduces reading/writing of intermediate files
  - may need more memory when several compiler phases are grouped

The blue folded-paper photograph is illustrative/decorative,
not a program example, state machine, or data-flow diagram.
No non-text operational relationship is encoded in the photo.

[SECTION NAVIGATION - CURRENT TOPIC]
Six-item Topics outline.
Highlighted item: 6. Types of compiler.
No diagram edges appear on the slide.

[FIVE COMPILER TYPE -> DEFINITION PAIRS]
1. One pass compiler
   -> compiles the process in one pass.
2. Two pass compiler
   -> compiles in two passes; generates intermediate code.
3. Incremental compiler
   -> recompiles changed source lines / code; updates object code.
4. Native code compiler
   -> target platform is the same kind as the host platform.
5. Cross compiler
   -> target platform differs from the host platform.

In the original slide each compiler type in a LEFT rounded box
is horizontally paired with its explanation in a RIGHT box.
The rows do NOT form a top-to-bottom flow or causal chain.